

Session 8: GitHub Agent Setup Steps

Step 1 - Create a GitHub Repository

The agent needs a target repository to work on. This should be a repository you own, with some content already in it (not completely empty).

1. Go to <https://github.com> and sign in.
2. Click the + icon in the top-right corner and select New repository.
3. Give the repository a name, for example: my-agent-demo.
4. Add a README file by checking the box that says Add a README file. This ensures the repo is not empty.
5. Click Create repository.
6. Note your repository path in the format username/repository-name. You will need this later.
Example: john-doe/my-agent-demo.

Important: Do not use a repository that contains real work or sensitive data. The agent will create branches and raise pull requests on whichever repo you point it at.

Step 2 - Create a GitHub App

The agent authenticates with GitHub through a GitHub App, not a personal access token. You need to create one under your account's Developer Settings.

1. Go to <https://github.com/settings/apps> and click New GitHub App.
2. GitHub App name: **Enter any unique name, for example: my-multi-agent or mas-github-agent22. The name must be globally unique on GitHub.**
3. Homepage URL: **Enter any valid URL. You can use your GitHub profile URL: <https://github.com/your-username>.**
4. Webhooks: **Uncheck the Active checkbox under Webhook. You do not need webhooks for this project.**
5. **Permissions — set the following under Repository permissions:**
 - Contents:** Read and write
 - Issues:** Read and write
 - Pull requests:** Read and write
 - Metadata:** Read-only (set automatically)
6. Click **Create GitHub App** at the bottom of the page.

7. On the next screen, find the **App ID** near the top of the page. Write it down you will need it for your .env file.

Note: You do not choose which repositories the App can access on this page. That is done when you install the App in Step 4.

Step 3 - Generate a Private Key

Still on the GitHub App settings page from Step 2, scroll down to the Private keys section.

1. Click Generate a private key.
2. GitHub will automatically download a .pem file to your computer. The filename looks like: your-app-name.2026-06-20.private-key.pem
3. Move this file somewhere easy to find, such as inside the cloned project folder (you will clone the project in Step 6). A good location is the resources/ folder inside the project.
4. Note the full file path to this .pem file. You will paste it into your .env file later.

Important: Do not rename the .pem file and do not open or edit it. Any modification will corrupt the key and cause a `jwt.exceptions.InvalidKeyError` at runtime.

Step 4 - Install the App on Your Repository

Creating the App is not the same as installing it. You must explicitly install it on your target repository, otherwise the agent will get a 404 or 403 error when it tries to access the repo.

1. Go to <https://github.com/settings/apps> and click your app name.
2. In the left sidebar click Install App.
3. Click Install next to your GitHub account.
4. Under Repository access, choose Only select repositories.
5. Select the repository you created in Step 1.
6. Click Install (or Save).
7. Verify: you should see a green confirmation and your repository listed under installations.

Step 5 - Get a Groq API Key

The agent uses Groq to run the LLM calls. You need a free API key.

1. Go to <https://console.groq.com> and sign up or log in.
2. Navigate to API Keys in the left sidebar.
3. Click Create API Key, give it a name, and copy the key.
4. Store the key somewhere safe you will add it to your .env file in the next step.

Note: Groq's free tier has token-per-minute limits. If the agent fails with a rate limit error during the session, try switching to a model with a higher limit, such as llama-3.3-70b-versatile or some frontier models

Step 6 - Clone the Project and Install uv

1. Clone the GitHub Agent repository:

```
git clone https://github.com/tnahddisttud/github-agent
cd github-agent
```

2. Install uv if you do not have it. Visit <https://docs.astral.sh/uv/getting-started/installation/#standalone-installer> and follow the instructions for your operating system.
3. Run uv sync to install all project dependencies:

```
uv sync
```

Step 7 - Configure the .env File

Inside the cloned project folder you will find a file called sample.env. Do not edit that file directly - copy it to a new file called .env and fill in your values.

On Mac / Linux:

```
cp sample.env .env
```

On Windows (Command Prompt):

```
copy sample.env .env
```

Open .env in a text editor and set these values:

GITHUB_APP_ID

The numeric App ID from Step 2. Example: 123456

GITHUB_APP_PRIVATE_KEY

The full absolute path to the .pem file from Step 3. Mac/Linux example: /Users/yourname/github-agent/resources/my-agent.pem Windows example: C:\Users\yourname\github-agent\resources\my-agent.pem

GITHUB_REPOSITORY

Your repository in the format username/repository-name. Example: john-doe/my-agent-demo

Do NOT use just the repo name without the username.

GITHUB_BASE_BRANCH

The name of the default branch of your repository. Usually: main (or master for older repos)

GROQ_API_KEY

The API key you copied in Step 5.

Critical: After editing, save the file with Ctrl+S (Windows) or Cmd+S (Mac) before running the agent. This is the single most common mistake the file is left unsaved and the agent reads empty values.

Step 8 - Run the Agent

From inside the github-agent directory, run:

```
uv run python -m src.agents.graph
```

Do NOT run the file directly with `uv run src/agents/graph.py`. That form breaks Python's relative imports and causes an `ImportError`. Always use the `-m` flag with the module path.

A `DeprecationWarning` about langchain-community at startup is expected and harmless. Wait for the agent to finish — it may take one to three minutes on the first run. Check your GitHub repository for a new branch and a pull request when it completes.

Common Errors and Fixes

Error: `ImportError: attempted relative import with no known parent package`

Fix: You ran the file directly instead of as a module. Use: `uv run python -m src.agents.graph`

Error: `github.GithubException.UnknownObjectException: 404 Not Found`

Fix: Your `GITHUB_REPOSITORY` value is wrong. It must be username/repository-name, not just the repo name. Also check that the `.env` file is saved.

Error: `github.GithubException.UnknownObjectException: Branch not found`

Fix: `GITHUB_BASE_BRANCH` does not match your repository's actual default branch. Check your repo on GitHub — the default branch is shown in the dropdown near the top of the file list (usually `main` or `master`).

Error: `jwt.exceptions.InvalidKeyError: Could not parse the provided public key`

Fix: The `.pem` file path in `GITHUB_APP_PRIVATE_KEY` is wrong, the file has been edited or renamed, or the path uses the wrong slash direction on Windows. Paste the full absolute path and check that the file is intact.

Error: `403 Resource not accessible by integration`

Fix: The GitHub App does not have write permissions on the repository. Go back to your App settings, check that Contents, Issues, and Pull Requests are all set to Read and write, then update the installation.

Error: `pydantic_core.ValidationError: Did not find github_repository`

Fix: The GITHUB_REPOSITORY variable is missing or empty in your .env file. Add it in username/repository-name format and save the file.

Error: groq.APIStatusError: 413 tokens per minute rate limit exceeded

Fix: Your Groq free-tier TPM limit is too low for the model you are using. Switch to a model with a higher limit in graph.py (e.g. llama-3.3-70b-versatile) or disable any active VPN, which can cause Groq to apply stricter limits.

Error: Branch created but no pull request

Fix: The create_pull_request tool is not included in the get_documentation_tools method in src/utils.py. Add it to the list alongside the other tools and rerun.